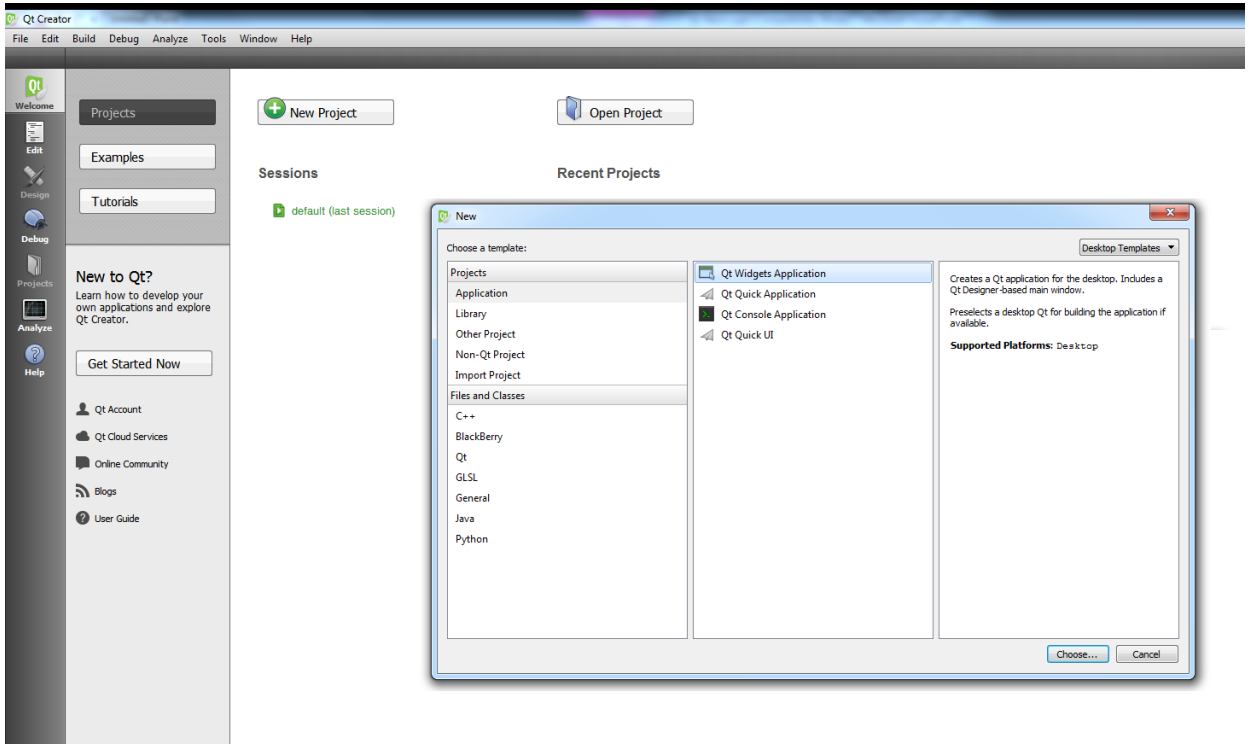
Qt-Basics

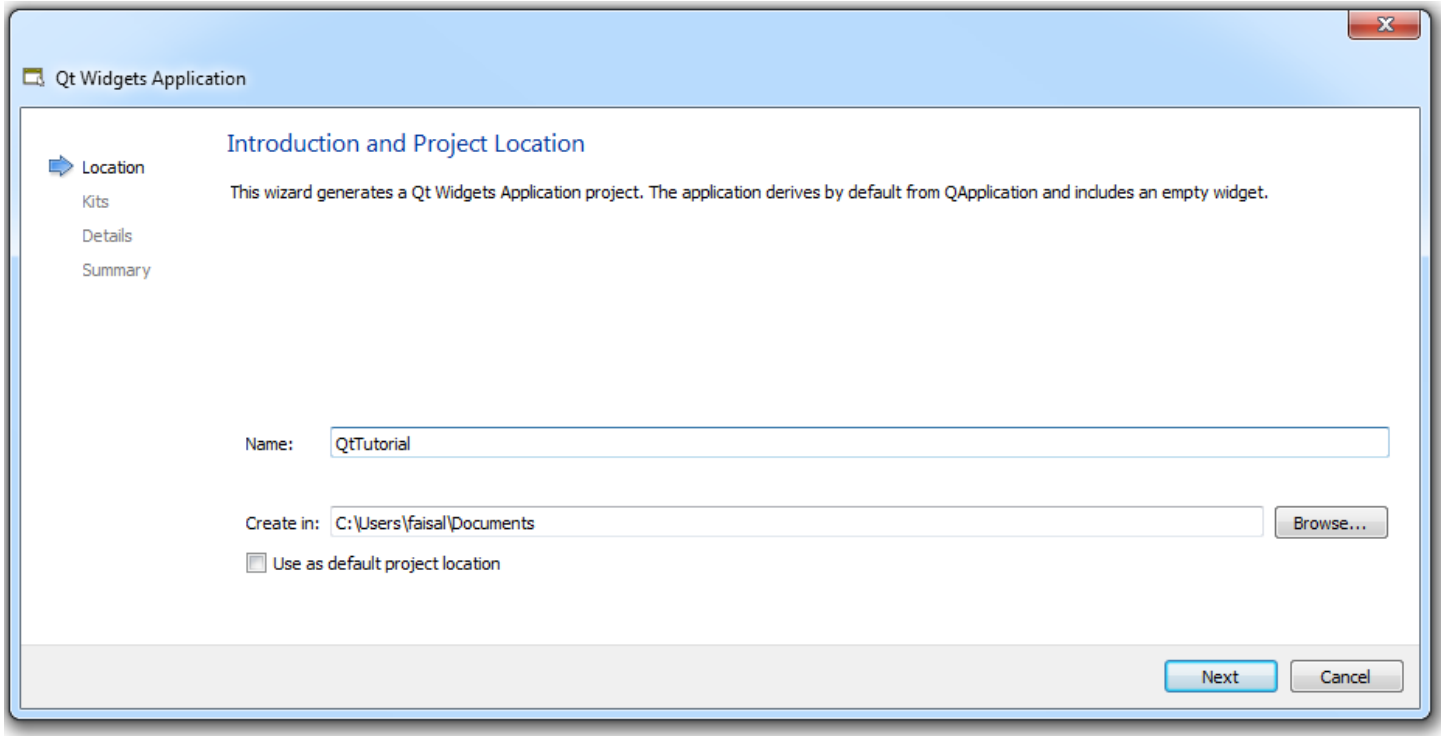
1. Launch the Qt-Creator-

2. Click on File -> New –>Qt Widget Application.



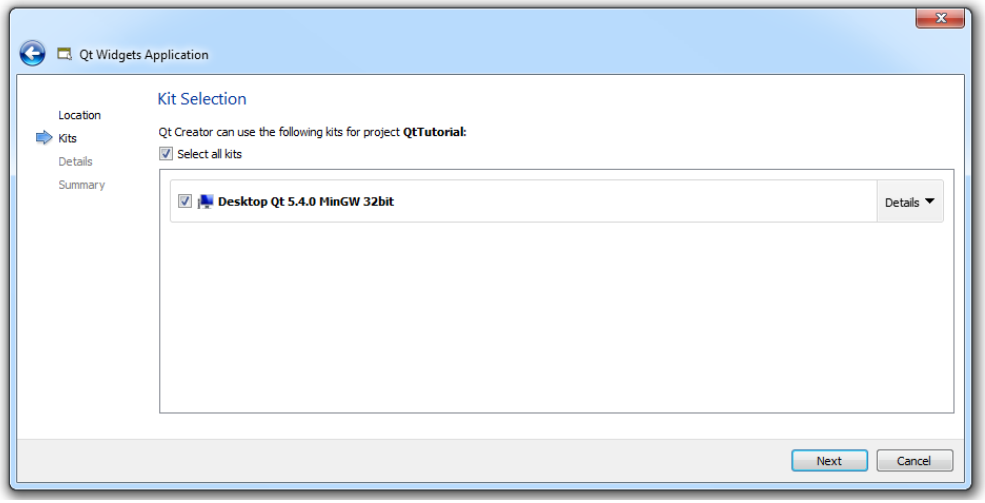
Qt-Basics

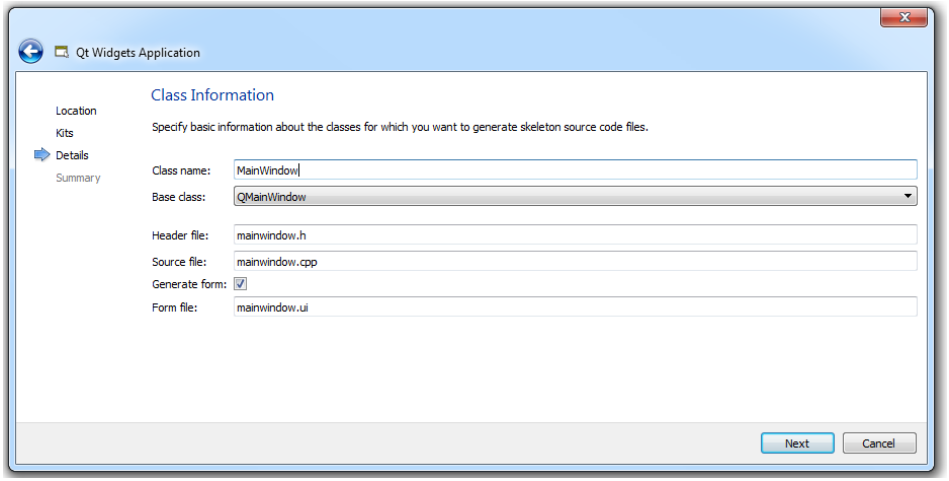
3. Name the Application and click Next.



Qt-Basics

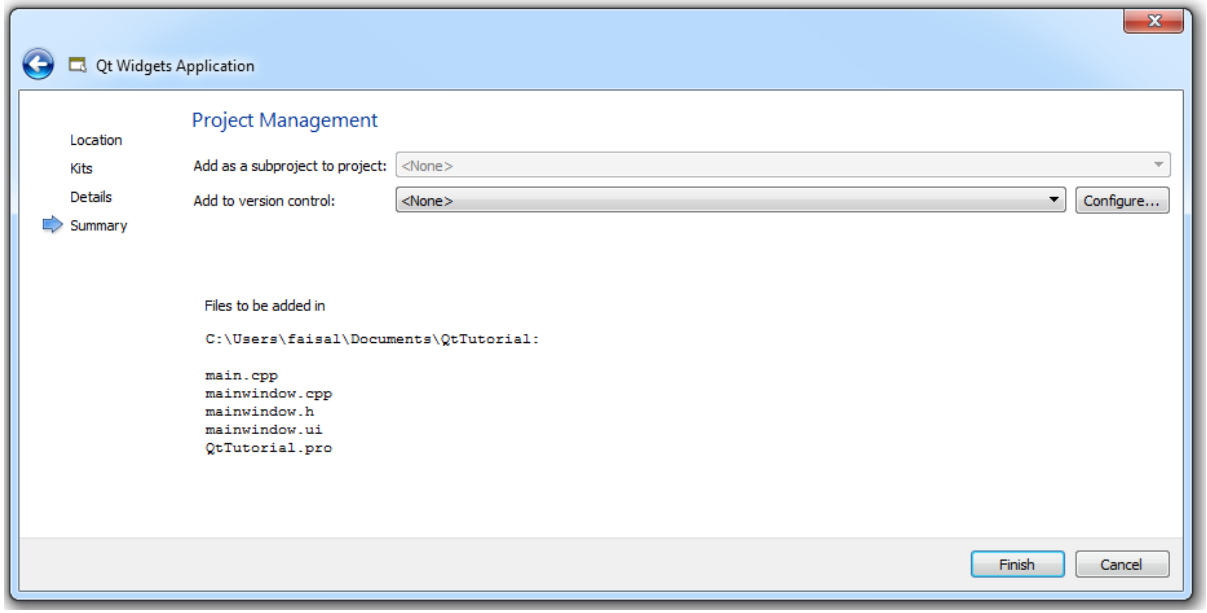
4.Click Next on both of the message as shown in the pictures:





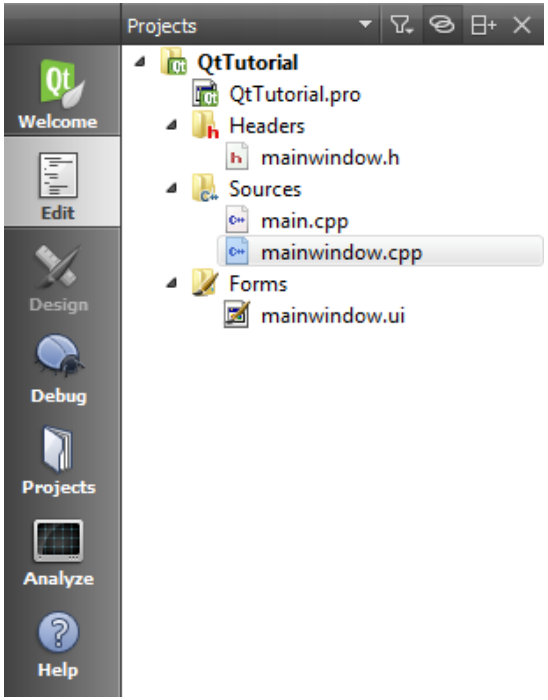
Qt-Basics

5. Click Finish.



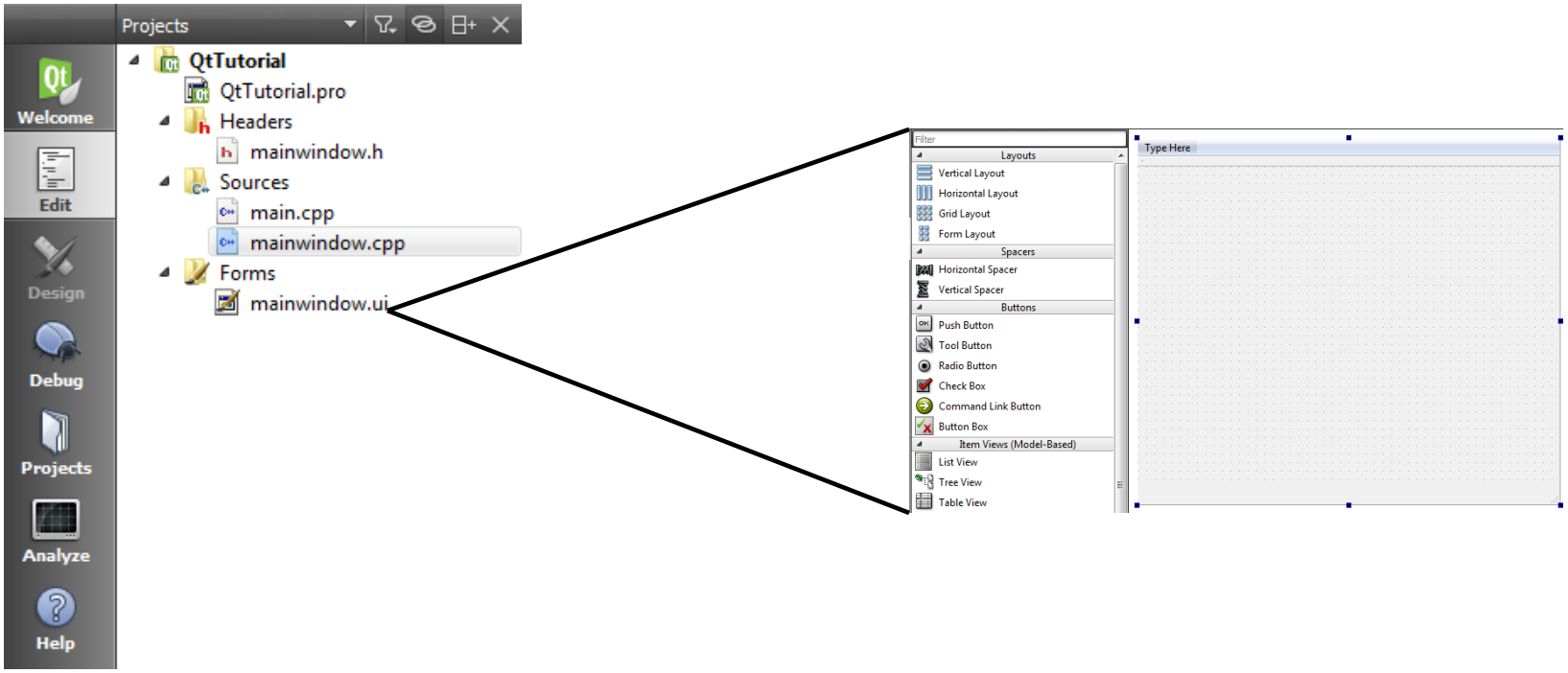
Qt-Basics

6. Following files will appear in the projects window.



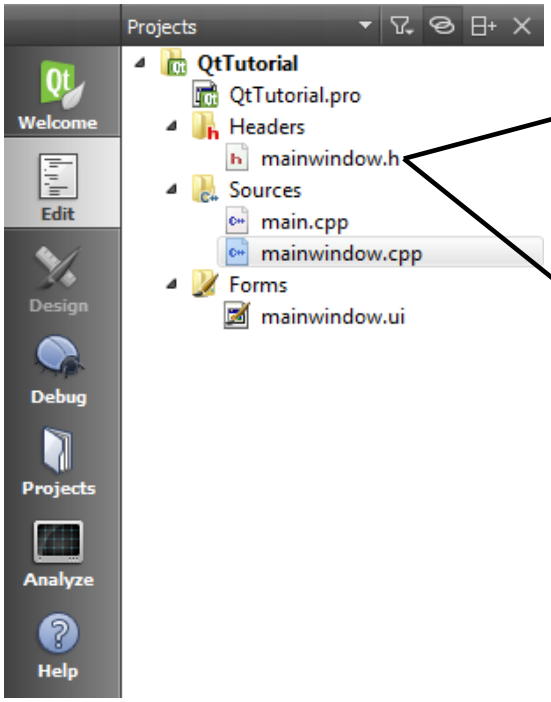
Qt-Basics

7. mainWindow.ui file is the design file where you put all the related controls. like button, radioBox, checkBox, TextBox….etc.



Qt-Basics

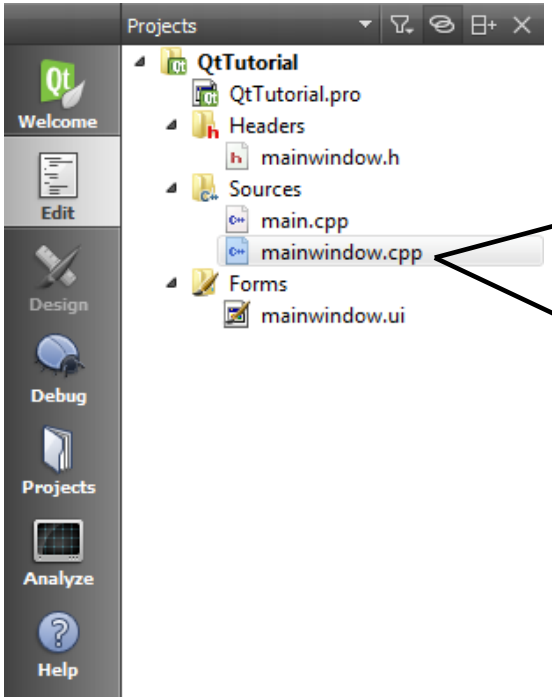
8. mainwindow.h file contains all the declaration of normal C/C++ functions and some special functions (signals and slots).




```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>

namespace Ui {
class MainWindow;
}

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = 0);
    ~MainWindow();

private:
    Ui::MainWindow *ui;
};

#endif // MAINWINDOW_H
```

Qt-Basics

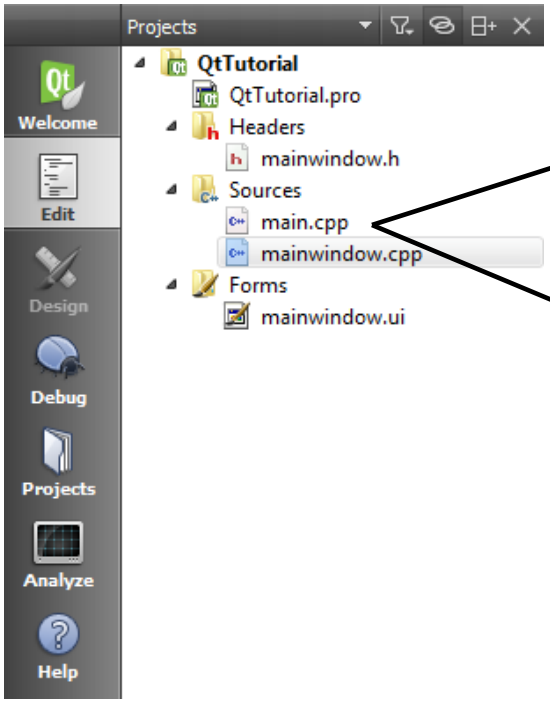
9. mainwindow.cpp file contains all the definition of the functions .




```
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

Qt-Basics

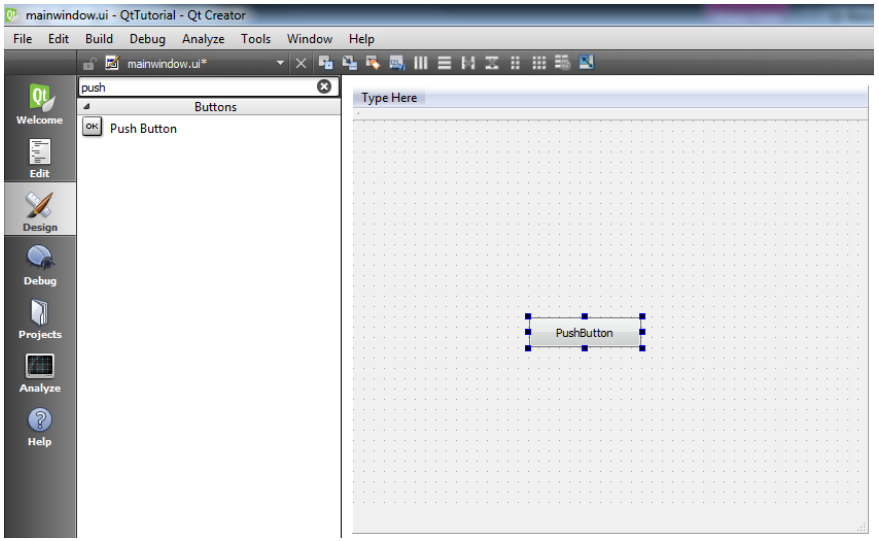
10. main.cpp files contains the object of the MainWindow class and we call its 'show()' function to launch the application.




```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();

    return a.exec();
}
```

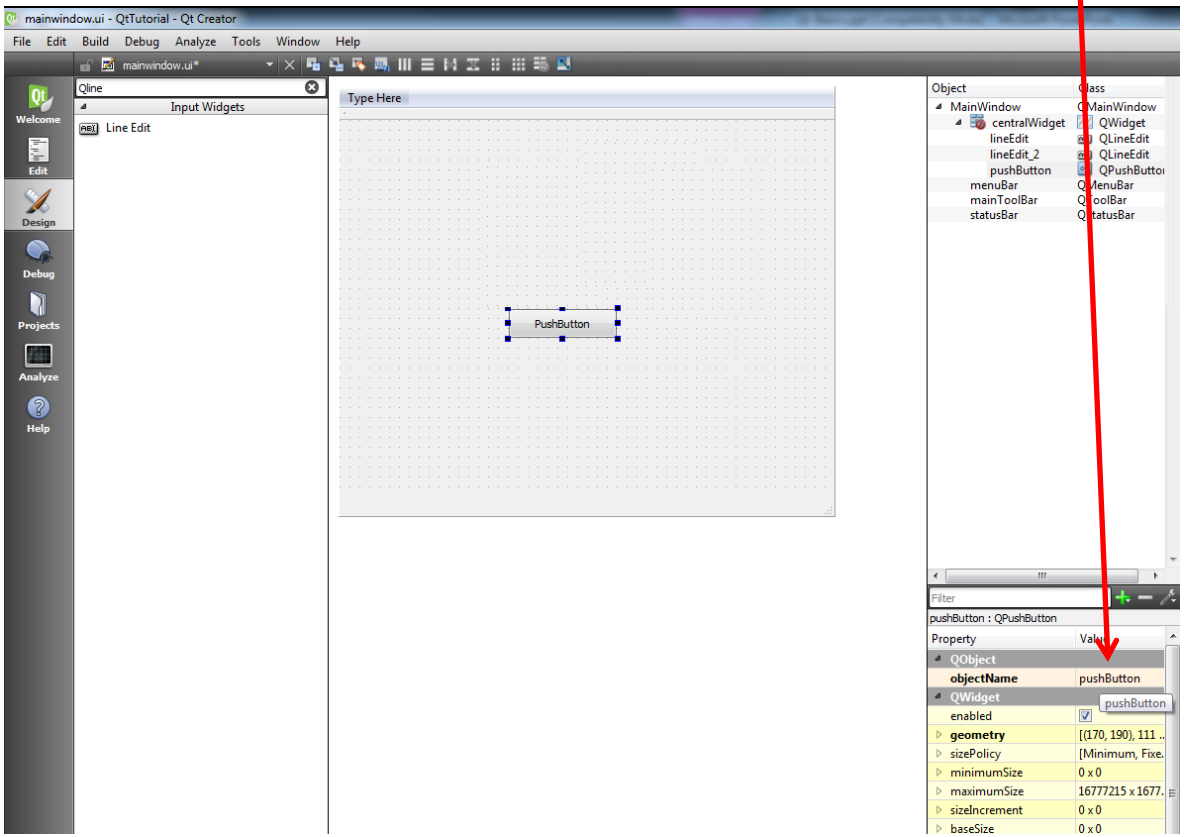
Qt-Basics

11. Click on mainWindow.ui file, following window will appear in the searchbox Type pushbutton, the pushButton will appear in the window below, drag the button and put it on the form.



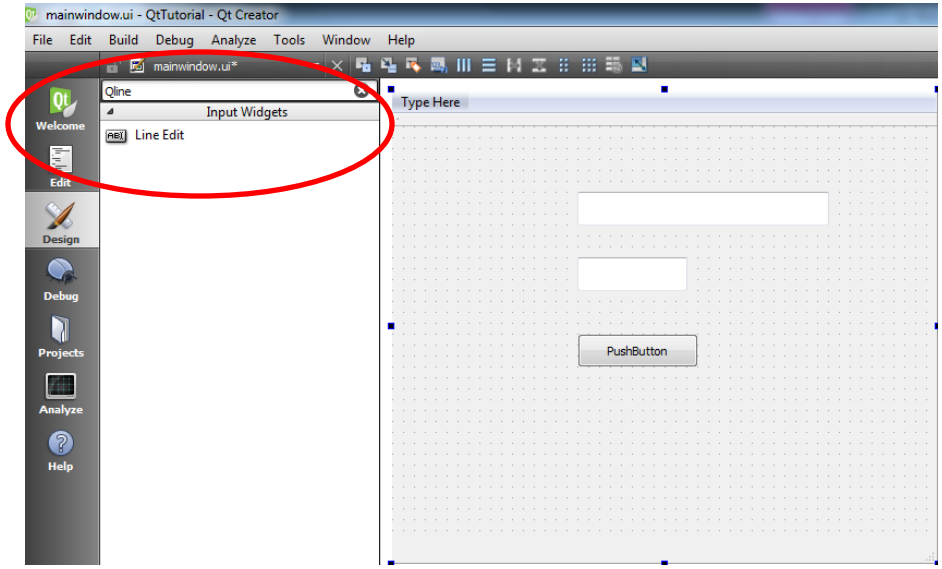
Qt-Basics

12: Each object (Qt widget) you put on the form has a name too, which is dispalyed here. You can change the name to pushButton as shown below.



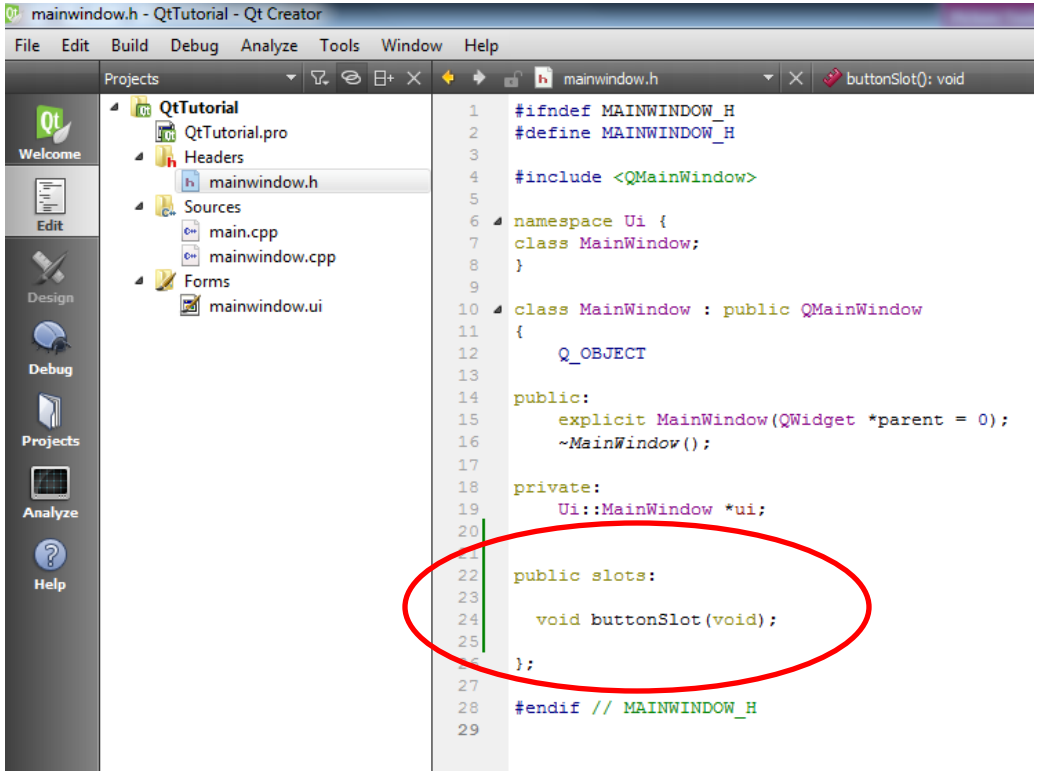
Qt-Basics

13: Type QlineEdit in the search box and put the line Edit (textbox) controls on the form and put two QlineEdit boxes on the form as shown below. Name one box textbox1 and other textbox2.



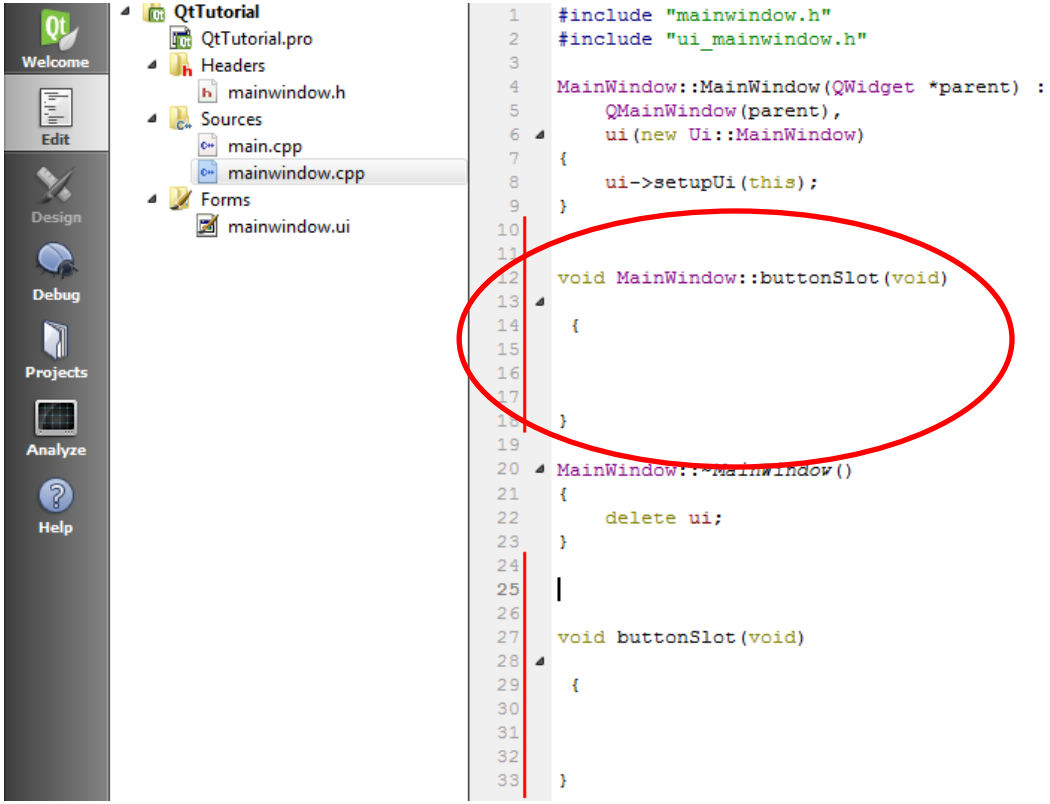
Qt-Basics

14: We want a function to be executed when we click the pushButton that's why
In mainWindow.h file It has be to declared in the public slots access specifier as shown below.



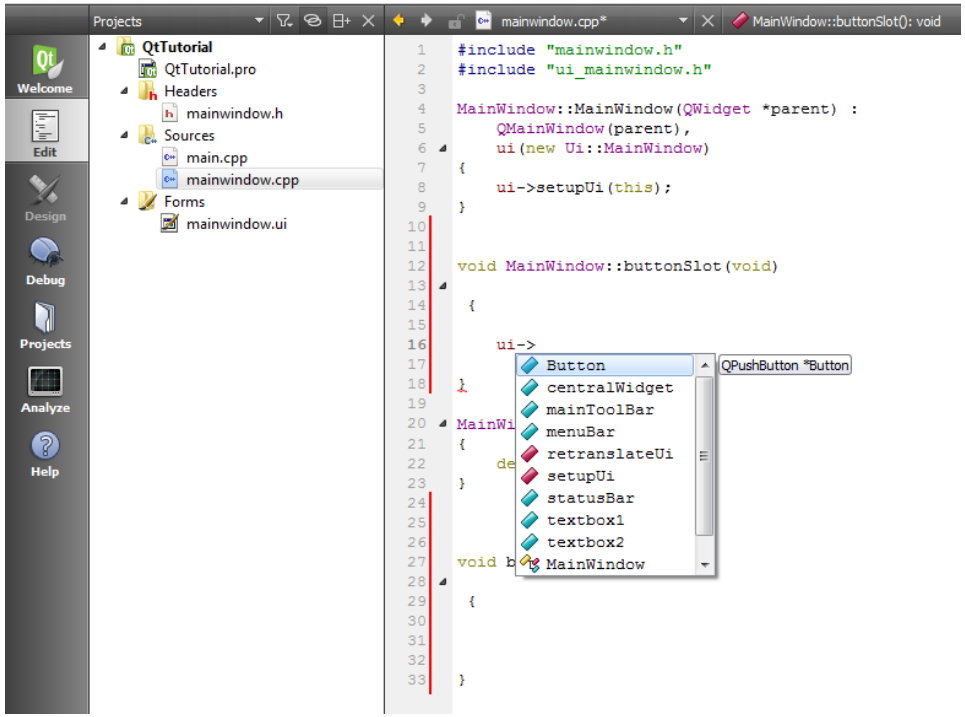
Qt-Basics

15: Define the function (slot) in mainwindow.cpp file .



Qt-Basics

16: When we click the button we want the text “Button is pressed” to appear in textBox1. textBox1 is present in .ui file that’s why it has to be called through ui (object of interface class)



Qt-Basics

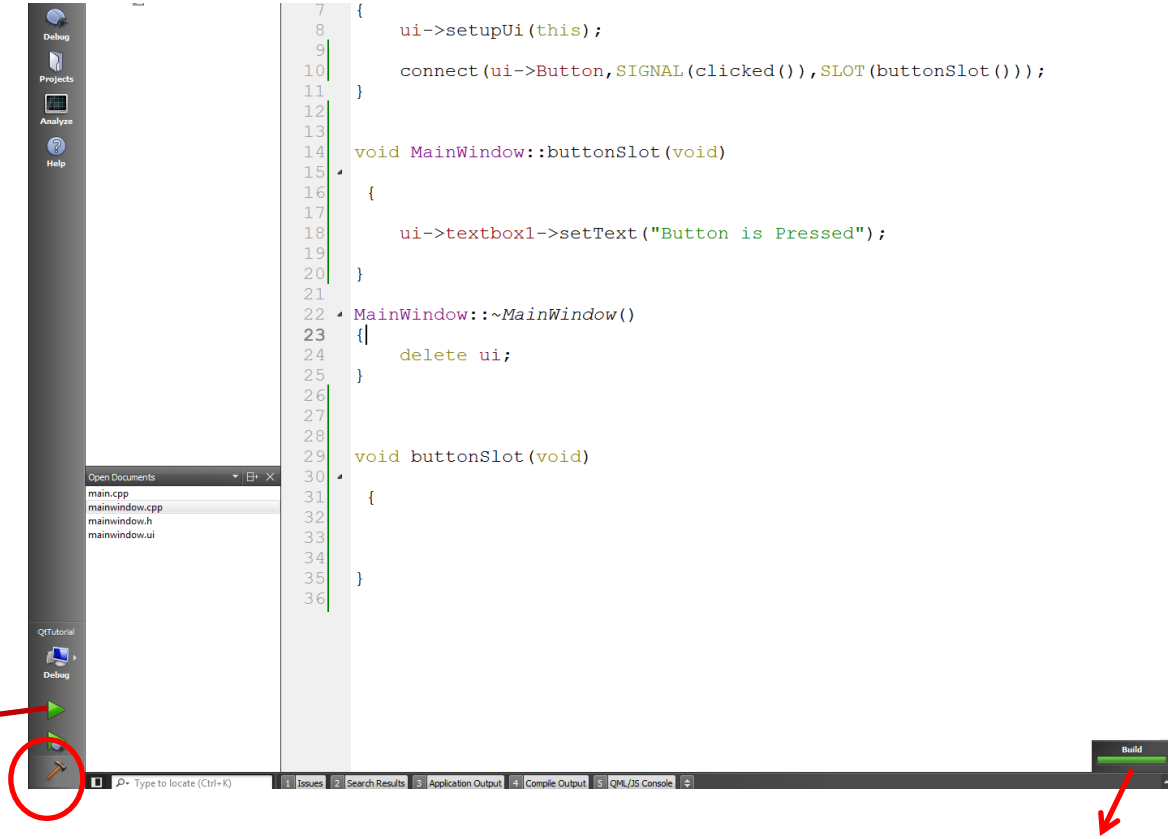
17: Call the text with setText() function of textbox1 object.

```
void MainWindow::buttonSlot(void)
 {
    ui->textbox1->setT
}
```

- setTabOrder
- setText — void setText(const QString &)
- setTextMargins
- setToolTip
- setToolTipDuration

Qt-Basics

18: Set the text in the the setText function,compile the code and execute as shown below:




```
7   {
8       ui->setupUi(this);
9
10      connect(ui->Button,SIGNAL(clicked()),SLOT(buttonSlot()));
11  }
12
13
14  void MainWindow::buttonSlot(void)
15
16   {
17
18      ui->textbox1->setText("Button is Pressed");
19
20  }
21
22  MainWindow::~MainWindow()
23  {
24      delete ui;
25  }
26
27
28
29  void buttonSlot(void)
30
31   {
32
33
34
35  }
36
```

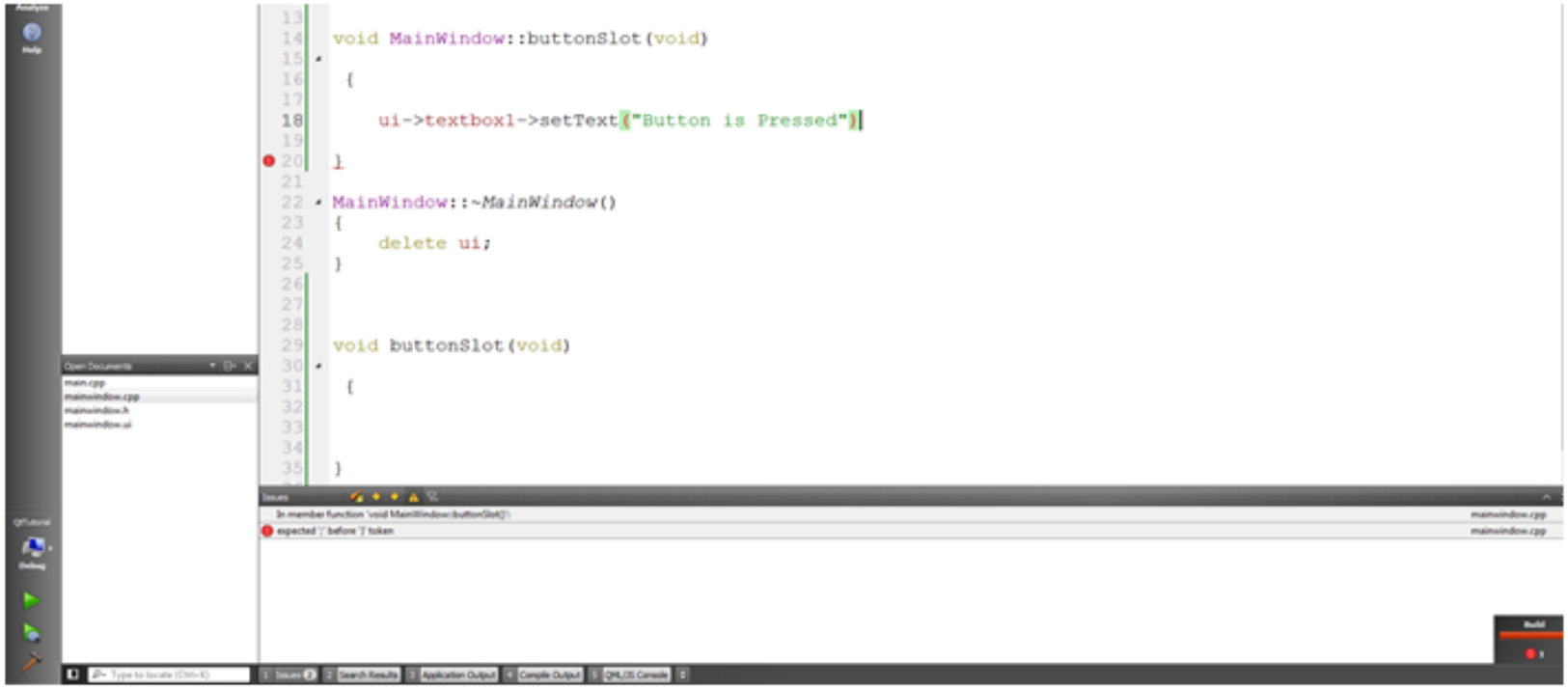
Run

Click to compile.

Compilation is successful without erorr.

Qt-Basics

19: If there is any mistake in the code the compilation error will be displayed
As shown below:



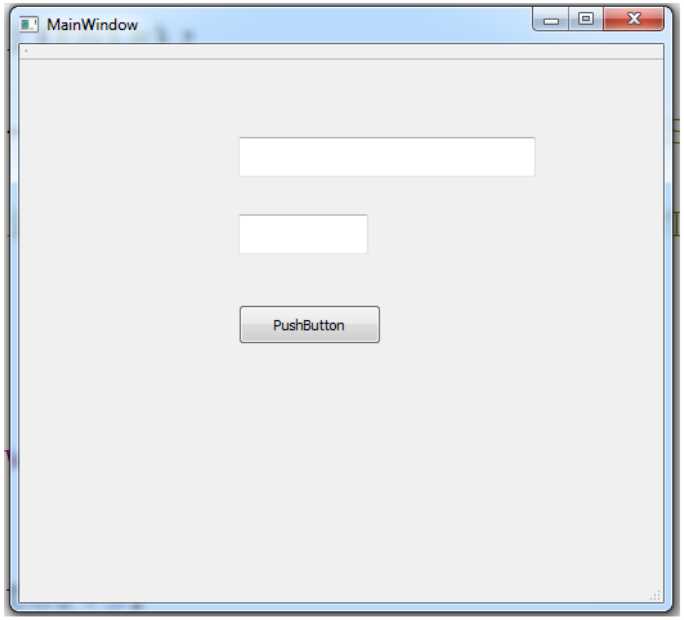
Qt-Basics

20: Connect the click event of the pushbutton with the buttonSlot function as given below, as a result of this connect statement every time when button is clicked the buttonSlot() Function will be executed.

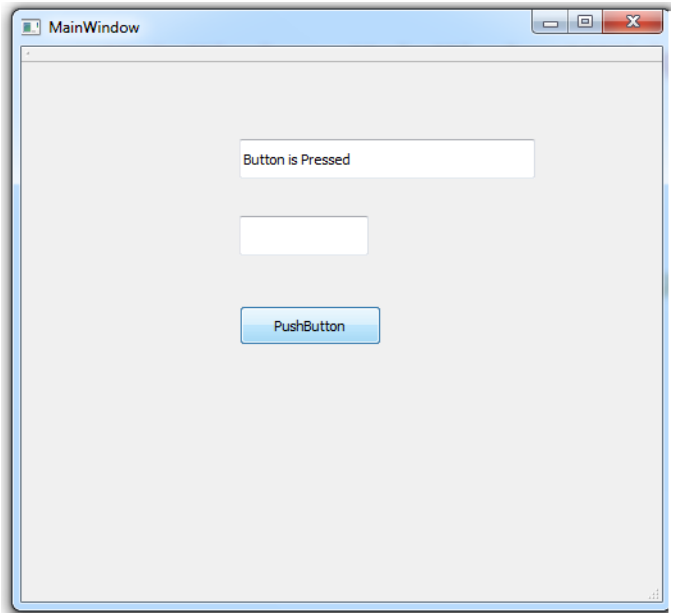
```
MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    connect(ui->Button,SIGNAL(clicked()),SLOT(buttonSlot()));
}
```

Qt-Basics

21: Now execute the code by clicking on the RUN button
Following windows will appear as the result of execution:



PushButton is pressed



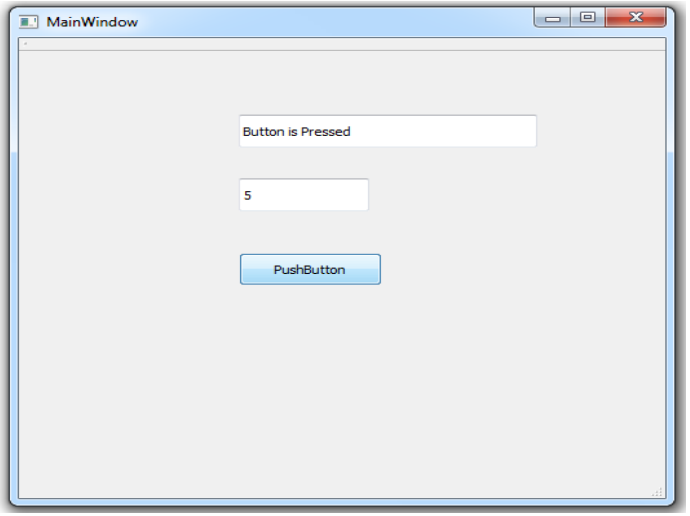
Qt-Basics

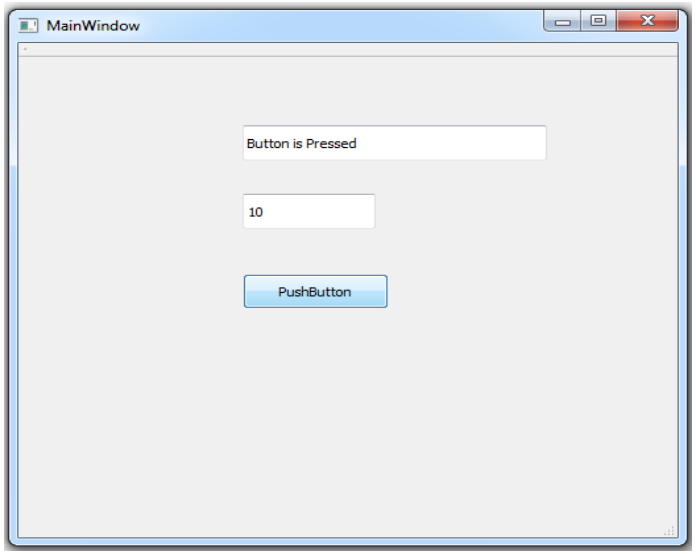
22: Update the textbox2 with the integers on each button click.
By default textBox can show the string message only that's why to put some other data type you have to do a Qt style typecasting as shown in the code below:

```
int factor=0;

void MainWindow::buttonSlot(void)

 {
    factor=factor+5;

    ui->textbox1->setText("Button is Pressed");

    ui->textbox2->setText(QString::number(factor));

}
```

Qt-Basics

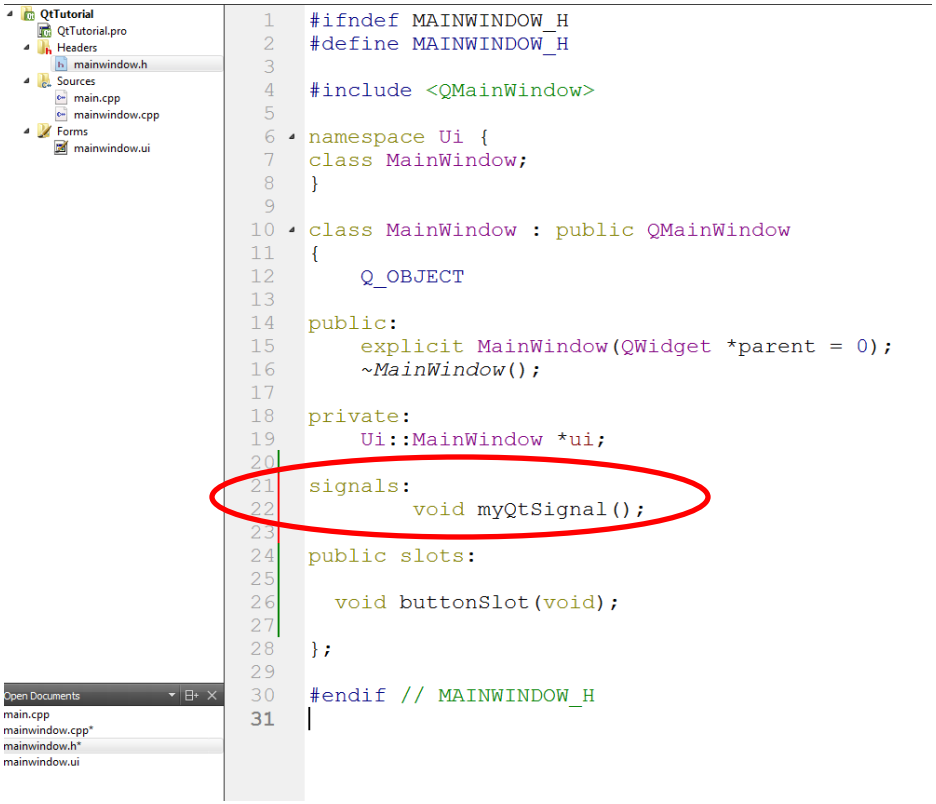
23: Whenever you click on the pushbutton
The first textbox will show the string and
The second will add 5 in textBox2.





Qt-Basics

24: Now we want to declare a signal as shown in the picture



Qt-Basics

25: We would like to connect the newly
Created signal with a new slot so declare
This slot (MesageSlot) too as shown in the picture:

mainwindow.h* MessageSlot(): void

```
1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3
4  #include <QMainWindow>
5
6  namespace Ui {
7  class MainWindow;
8  }
9
10 class MainWindow : public QMainWindow
11 {
12     Q_OBJECT
13
14 public:
15     explicit MainWindow(QWidget *parent = 0);
16     ~MainWindow();
17
18 private:
19     Ui::MainWindow *ui;
20
21 signals:
22         void myQtSignal(void);
23
24 public slots:
25
26   void buttonSlot(void);
27
28   void MessageSlot(void);
29
30
31 };
```

Qt-Basics

26: Define the slot MessageSlot as shown in the mainwindow.cpp file.
We want a messagebox to appear with the text ‘test’ whenever this function exectues.

```
void MainWindow::MessageSlot(void)

{
    QMessageBox *msgBox = new QMessageBox(this);
    msgBox->setText("Test");
    msgBox->exec();

}
```

Qt-Basics

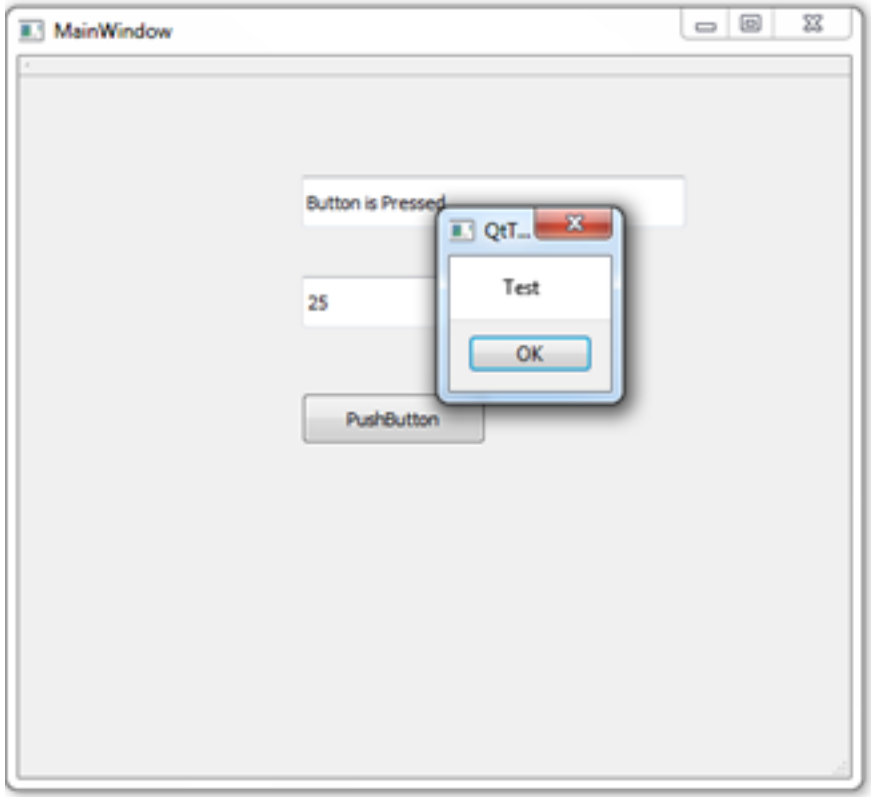
27: Connect the signal ‘myQtSignal’ with the MessageSlot.

```
#include "mainwindow.h"
#include "ui_mainwindow.h"
#include"QMessageBox"

MainWindow::MainWindow(QWidget *parent) :
    QMainWindow(parent),
    ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    connect(ui->Button,SIGNAL(clicked()),SLOT(buttonSlot()));

    connect(this,SIGNAL(myQtSignal()),SLOT(MessageSlot()));
}
```

Qt-Basics

28: We want to emit the signal 'myQtSignal'whenever the value of textbox2 is =25 that's why update The buttonSlot() function:

```
void MainWindow::buttonSlot(void)

 {
    factor=factor+5;

    ui->textbox1->setText("Button is Pressed");

    ui->textbox2->setText(QString::number(factor));

    if(factor==25){

    emit myQtSignal();

    }
```

Qt-Basics

29: Run the code with the following expected result: